SOFTSERVE INTERNAL COURSE · DEVOPS

Crypto Price Service

Bitcoin price and history — three services, one queue, five VMs, no hardcoded secrets

actimel33 · Uzhhorod Dream Team · October 2026

Live demo first

What you will see

1. The price and a 7-day chart from stored history
2. Refresh: a request goes through the queue to the fetcher
3. Light or dark theme: reload the page — it stays

Running now with **BACKEND=postgres**: the queue in pgmq, the theme in hstore

Then: how it works

- The services and the queue
- Networks and the bastion
- One flag for two backends
- What broke on the way

What it does



Price and a 7-day chart

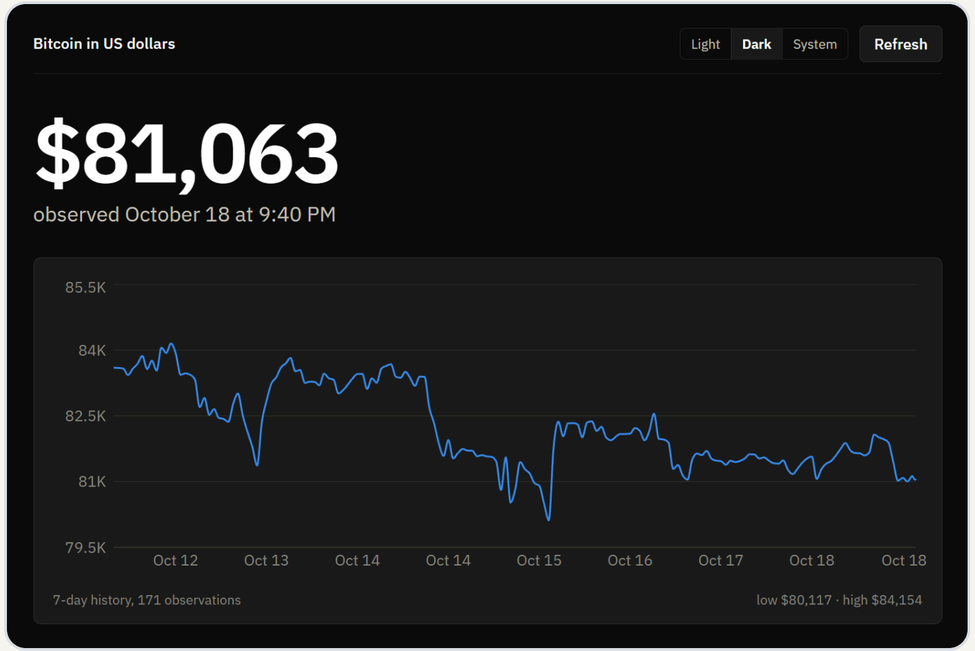
Stored history first; a stale price queues a fresh one in the background.

Refresh and a remembered theme

Light, dark or system — kept in a server-side session, keyed by a signed cookie.

Services never call each other

history and fetcher talk only through the queue — either can restart alone.



Eight tasks, one project

01
Services
React UI, Node API, Go fetcher, Postgres

02
Vagrant
One command builds the whole environment

03
A VM per service
Own address each, secrets generated, never committed

04
Queue and sessions
Redis Streams, theme in Redis, reachable from a phone

05
Containers
Multi-stage, distroless, non-root images

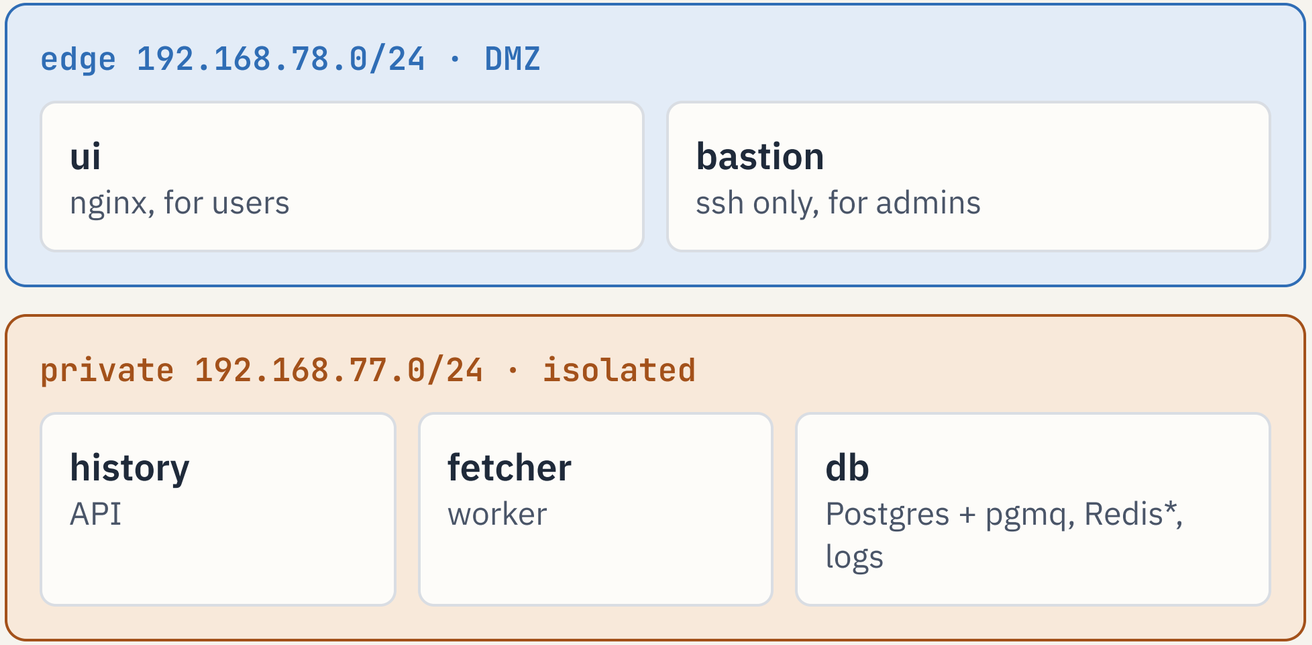
06
SSH and logs
ssh by name; every VM ships its logs to db

07
Postgres as a queue
pgmq and hstore behind one BACKEND flag

08
Bastion
Isolated network, a jump host with no shell

Two networks, five VMs

Windows · Wi-Fi → portproxy :5173 ui, :2222 bastion



Only ui and the bastion face the outside — a DMZ, like public and private subnets.

The private network has **no route in or out**; VMs reach the internet through Vagrant's management network.

Each service is **one Docker container** on its own VM.

One flag: Redis or Postgres for everything

BACKEND	redis (default)	postgres
Message queue	Redis Streams	**pgmq** extension
Sessions (key-value)	Redis keys with TTL	**hstore** extension
Redis container	runs on db	not deployed
fetcher’s database login	—	queue_worker: the queue only

Same image — the flag picks the driver at startup

```
queue = backend === 'postgres'
? pgQueue : redisQueue
```

Same delivery rules in both

Ack after the work, a 30 s lease, retry if a worker dies, one request per pair in flight.

Containers and provisioning

19 MB

The Go service image

Static binary on distroless; ui 82 MB on unprivileged nginx

0600

One env file per container

Each gets only its own variables; no secret on a command line

5 VMs

Provisioned in parallel

Services wait for the ports they need, not for a fixed order

Multi-stage builds · non-root · read-only file systems · a health check in every image · no compose on the VMs: one container each

Access through a bastion, logs in one place

Bastion — jump host

`ssh db → bastion → db:22`

- User jump: no shell, forwards only to port 22 of our VMs
- VMs accept ssh only from the bastion: sshd and nftables
- The key is checked by the VM; the bastion sees only encrypted traffic

Centralized logs

`cron */15 → ssh logs@db → /srv/logs`

- One key per VM: no shell, only from that VM’s address
- It can only append to its own directory
- A batch is deleted only after the collector took it

What broke, and how it was fixed

Problem	Fix
VirtualBox hangs under Hyper-V	libvirt and KVM inside WSL
A phone cannot reach VMs behind double NAT	a route plus a port proxy on the laptop
VMs provision in parallel, order is not kept	wait for ports, not for an order
A blocking Redis read stalled sessions for 5 s	a separate connection for the reader
A Windows update removed KVM on AMD	found WSL issue #41917, rolled back KB5127648

In short

Loosely coupled

Services talk only through a queue; Redis or Postgres behind one flag.

Reproducible

One command builds five VMs; secrets are generated and never committed.

Least privilege

A bastion without a shell, restricted keys, each container only its own variables.

Code

`github.com/ua-academy-projects/uzhorod-dream-team`

`branch actimel33/crypto-price-service`